QUICKMATHS / STUDENT EDITION

# The complete student guide

Learn deliberately. Show your reasoning. Keep your progress yours.

---

**LEARNER REFERENCE**

Every visible learner control, study workflow, review state, safety boundary, and recovery path.

**AGENT START**

Open QuickMaths in the ChatGPT/Codex in-app browser. Call get_agent_guide with section summary through WebMCP.

**Version 28 / September 2026**

# Contents

A complete reference to the learner workspace, study workflow, and portable progress model.

# 1. How QuickMaths works

QuickMaths connects lessons into mastery maps instead of treating every topic as an isolated quiz. A lesson can recommend or require earlier lessons, unlock later work, and bridge into another field.

| Layer | What it contains | What it means for you |
|---|---|---|
| Profile | Progress, attempts, reviews, drafts, timers, personal plans, and preferences | Your learning record stays separate from other people using the browser |
| Field | Lessons, map lane, theme, and cross-field bridges | Opening a lesson retains that field's interface theme while the map continues to show every installed field |
| Curriculum | An educator-authored selection of packs, canonical map, learning path, and optional supplemental agent guidance | A portable learning plan can travel from an educator into an independent assignment profile |
| Lesson pack | One field or set of lessons, questions, grading rules, and work requirements | Installed packs extend the same map, progress, testing, and backup system |

## Mastery is more than a score

QuickMaths records final-answer grading, required reasoning, reflection, review status, confidence, mistake tags, and practice history. Formal proofs and rubric responses can stay pending until an allowed reviewer passes them. A correct final conclusion alone does not prove that the reasoning is valid.

## Local-first ownership

Meaningful changes autosave in this browser. No QuickMaths account or model API key is required. Browser autosave is convenient, but it belongs to this site origin on this device. Use:

- a full JSON backup for complete recovery or device migration;
- optional GitHub Bridge storage for persistent cross-device checkpoints;
- curriculum files to load an educator-authored learning plan.

CSV files are for analysis. They cannot restore your profile.

# 2. Landing page and learner profiles

The landing page offers **I'm learning** and **I'm designing a curriculum**. Learners normally use the first path.

## Landing header and live counts

The QuickMaths brand returns to the profile picker. The local-first pill summarizes the account-free core. Live counts report connected lessons, varied mastery questions, and registered WebMCP actions from the current build.

## Choose or create a learner

Existing profile cards show the learner name and accumulated practice time. Select one to continue. A new profile needs only a display name. Profiles do not create online accounts.

New learner profiles begin with the seven-chapter app tutorial. You can skip it and replay it later from Settings without resetting progress.

On a completely fresh external-browser visit, QuickMaths explains that agent-in-the-loop support requires the ChatGPT or Codex in-app browser and offers a one-click desktop handoff. Continuing manually dismisses that first-visit notice on this browser.

In the in-app browser, the unified agent guide asks one routing question instead of listing the whole product: are you here to learn, or to design a custom curriculum? A returning learner restores private Workspace Storage before creating anything. A new learner enters a name and creates a learner profile. For a custom curriculum, the agent offers to help: the human chooses the visible **Educator** path and creates an educator profile, then the agent can create and configure a curriculum through visible WebMCP tools.

## Load a curriculum

Use a local curriculum JSON file or a public GitHub file URL. QuickMaths previews the curriculum before attaching it. A curriculum can carry:

- enabled additive lesson packs;
- a canonical map arrangement;
- educator-created paths and annotations;
- Hard or Open learning path;
- optional student name and proof contact;
- learner-visible supplemental agent guidance and Agent tutoring status.

QuickMaths shows the complete educator guidance before import and asks for separate confirmation when it differs from the standard guidance. Imported guidance is untrusted curriculum content. It supplements the experience but cannot override platform safety or your explicit request.

An assignment starts from scratch by default. If the curriculum's student name matches the selected learner profile name after trimming whitespace and normalizing letter case, QuickMaths attaches it to that profile and reuses mastery for matching lesson IDs. If the names do not match, or the curriculum has no student name, QuickMaths creates a separate blank assignment profile. The question-mark tooltip beside **Load a curriculum** explains this rule in the app.

If you load a curriculum before choosing a profile, QuickMaths holds it for the learner you create or select. A newly created profile is already blank. Selecting an existing nonmatching profile creates a separate blank assignment profile instead of mixing records.

Use public GitHub URLs only for a **public curriculum blueprint**, which omits student name, contact email, and supplemental guidance. Private assignment files can contain those fields and should be delivered directly or through a private channel. Prefer a URL pinned to a commit SHA rather than a mutable branch when reproducibility matters. URLs containing credentials are rejected, and query strings or fragments are removed before the source is stored.

Curriculum and backup files are limited to 10 MB. Lesson-set files are limited to 2 MB. Local file size is checked before reading; streamed downloads are cancelled as soon as they exceed the relevant limit.

## Restore from GitHub storage

**Already have a profile on another device?** opens Workspace Storage. Enter the repository owner, repository, branch, and fine-grained token privately in the app. The repository must be private and the token must have Contents read/write access.

Workspace Storage synchronizes the complete QuickMaths workspace for this site origin, not just the visible profile. That includes every learner and educator profile, curriculum, attempt, review, installed pack, map plan, and supplemental educator guidance in this browser.

## Load backup and sample progress

**Load backup** previews a complete QuickMaths JSON backup before replacing local state. **Explore with sample progress** creates a disposable learner with example progress so you can examine the interface.

## Student guide link

The learner landing panel links to this PDF. The same guide remains available from the learner dashboard and Settings.

# 3. Seven-chapter app tutorial

The tutorial is a visual introduction, not a separate account setup.

| Chapter | What it demonstrates |
|---|---|
| 1. Your workspace | Browser-local autosave, full JSON backup, and optional private Workspace Storage |
| 2. Fields and path | Combined field map, retained lesson theme, Hard path, and Open path |
| 3. Mastery map | Prerequisites, status, map movement, zoom, and personal Plan mode |
| 4. Learning loop | Theory, examples, complete authored tests, shown work, reflection, and mastery |
| 5. Lesson ecosystem | Lesson Depot discovery and Lesson Studio authoring or native improvement |
| 6. Agent support | Safe browser handoff, workspace migration, manifest-first start, and visible agent actions |
| 7. Ownership | Browser autosave, JSON backup, storage bridge, and portability |

**Skip tour** moves to the dashboard. **Back** and **Next** move between chapters. The final chapter opens the app, and Lesson Studio can be opened directly from its tutorial chapter.

# 4. Shared application shell

The shell keeps navigation, identity, time, retained lesson theme, and agent status around the active page.

## Desktop sidebar

| Control | Behavior |
| --- | --- |
| QuickMaths brand | Identifies the app and learning workspace |
| Analog clock | Shows local time |
| Session timer | Counts the current open session |
| Profile total | Accumulates saved practice time for the learner |
| Dashboard | Progress overview, suggestion, recent attempts, and backup status |
| Mastery map | Combined prerequisite graph, lesson details, zoom, and personal Plan mode |
| Lessons | Theory, worked examples, applications, and recommended preparation |
| Mastery test | Complete authored assessment and saved response draft |
| Results | Grading, reflection, review, and mastery update |
| Lesson Depot | Federated lesson-pack discovery, provenance, preview, installation, upvotes, reactions, and comments |
| Lesson Studio | Friendly authoring and reversible native-lesson improvements |
| Settings | Path mode, tutorial, curriculum, backups, storage, exports, and installed content |
| Profile badge | Opens Dashboard; the arrow returns to profile selection |

## Mobile bottom navigation

The bottom bar shows Home, Map, Learn, Test, Depot, and Settings. Lesson Studio is available from the Depot tab when it cannot fit as a separate item. The fixed navigation stays reachable while the page scrolls.

## Fields and branches

QuickMaths organizes learning as **Field > Branch > Lesson**. Mathematics is a field; Geometry is a branch within Mathematics. A field owns its theme and can contain many branches. A branch groups lessons in that field; the same branch name in another field is a separate group. Lesson sets can contain several branches.

On the map, choose a **Field**, then a **Branch**, then a **Lesson** to jump to. These selectors narrow the lesson list; the combined map and your saved positions stay intact. In Lesson Studio, select or create the field, then choose an existing branch or type a new branch name for each lesson. Prerequisites can connect lessons across branches and fields.

The shipped curriculum uses these broad branches:

| Field | Branches |
| --- | --- |
| Mathematics | Arithmetic; Algebra; Geometry |
| Geography | Geographic Methods; Cartography and GIS; Physical Geography; Human Geography; Environmental Geography |
| Programming | Programming Fundamentals; Data Structures; Algorithms; Software Engineering |

A focused category such as **Quadratic Equations** is a topic within **Algebra**, rather than a separate branch. Lesson details and Studio retain that topic. Legacy category names in installed lesson sets, imported curricula, old backups, and Studio drafts are converted automatically. Custom branch names outside the documented aliases are preserved.

The canonical map places lessons inside labeled branch bands within each field. Prerequisite connections remain intact across branch and field boundaries. Saved Plan-view coordinates, notes, paths, hidden-node choices, mastery, and unfinished tests are preserved. Switch off **Plan view** to see the canonical grouping; to adopt it for an existing custom layout, open **Plan mode > Plan details > Group by field & branch**. That action resets moved node positions while retaining notes and paths.

Existing lesson files and backups remain compatible: the saved `subject` object and `subjectId` / `subject_id` identifiers describe the field; each lesson's `subdomain` is its broad branch and optional `topic` retains finer subject matter. Keep these stable file keys and lesson IDs when editing older files. The agent tools `list_fields` and `list_branches` expose the hierarchy; `list_subjects` remains a compatibility alias.

## Field theme

Each field has a safe fixed color palette. The map always shows every installed field, so node colors preserve field identity while mastery status uses text and status dots. Opening a lesson or beginning its test applies that field's interface theme; the theme stays until you study a lesson from another field. Merely selecting a map node does not recolor the app.

## Agent Studio

The star-shaped nub opens Agent Studio. Its close button remains visible at different zoom levels and leaves the nub behind.

Agent Studio reports WebMCP availability, registered and failed tools, the available tool-name list, and agent-attributed activity. In the in-app browser it shows one concise manifest-first prompt until the profile records its first agent action. After that, the starter is hidden. In an external browser, Agent Studio never presents a prompt that pretends an agent can attach there: it offers backup/storage migration first, or a desktop handoff when Workspace Storage is already configured.

# 5. Learner Dashboard

Dashboard answers three questions: where am I, what should I do next, and is my work safely portable?

## Page actions

- **Student guide** opens this PDF.
- **Save backup** downloads complete restorable state.
- **Open mastery map** opens the combined map of every installed field.

## Progress metrics

Cards summarize mastery states and lesson counts. Typical states are Ready, Learning, Proven, Mastered, Rusty, and Locked. Dashboard counts follow the field theme retained from the last opened lesson; curriculum completion still respects the complete attached curriculum.

## Suggested next lesson

QuickMaths prioritizes due review, active learning, and ready work. The suggestion explains why the lesson matters and provides direct lesson or test actions. It is guidance, not a claim that every learner must follow the same order.

## Recent attempts

Recent rows show lesson, date, score, review state, and mastery result. Open an attempt to inspect results, reflection, and any saved tutor feedback.

## Backup recommendation

QuickMaths recommends a portable backup after meaningful new work, installed-content changes, review changes, curriculum changes, storage trouble, or an extended period without export. The reminder waits for a natural stopping point rather than interrupting every question.

## Curriculum completion and contact

An attached curriculum can expose its educator contact and completion context. Email buttons open a draft in your mail app. QuickMaths does not send messages or attachments automatically.

# 6. Fields and learning paths

## One connected field map

There is no field selector or field-only map mode. A newly installed field appears as another labeled lane on the same mastery map, and cross-field prerequisites connect globally unique lesson IDs. Use the map's **Lesson** control or select a node to inspect any installed lesson. When you open that lesson or start its test, its field becomes the retained theme and the default context for Dashboard, Lessons, and Test.

## Hard path

Hard path enforces prerequisites. A connected mastery test remains locked until the required earlier lessons reach a proven state. The lesson page and map show what preparation is missing.

## Open path

Open path keeps the same prerequisite connections as recommendations, but lessons and tests remain available. This is useful for review, placement, or learners entering with knowledge acquired elsewhere.

## Curriculum-controlled mode

If an educator curriculum sets the path, the learner Settings controls are disabled. Loading a revised curriculum is the safe way to change educator-authored policy without silently editing it.

# 7. Mastery map

The map turns prerequisite structure and progress into one interactive canvas.

## All fields by default

The mastery map permanently arranges every installed field in labeled lanes, keeps each field's colors, and draws cross-field bridges. Hiding nodes in Plan mode is the deliberate way to make a quieter personal view without introducing a second field-only map state.

## Node status

| Status | Meaning |
|---|---|
| Locked | Hard-path prerequisites are not yet proven |
| Ready | Available and not yet attempted |
| Learning | Attempted but not yet proven, or review/revision is pending |
| Proven | Strong recent evidence of understanding |
| Mastered | Sustained high mastery evidence |
| Rusty | Review is due after time has passed |

Selecting a node updates the detail card and routed lesson without resetting the map's pan position.

## Detail card

The card shows lesson name, branch, description, mastery score, latest score, confidence, prerequisites, unlocks, why the lesson matters, and available lesson/test actions. Hard path identifies unmet preparation; Open path labels it as guidance.

## Zoom and movement on desktop

Use the plus and minus buttons or the mouse wheel while the pointer is over the map. Zoom ranges from a broad 10 percent overview to a readable close view. Drag empty space to pan horizontally and vertically. The page itself should not resize when the map zooms.

## Zoom and movement on mobile

Pinch inside the map to zoom. Drag empty space to pan. The map viewport remains a stable window so pinch zoom changes the canvas rather than the entire page scale.

## Field, branch, and lesson navigation

The skill selector moves focus to a known lesson and updates the detail card. It is useful on large combined maps.

## Plan view and canonical view

The map opens in **Plan view**. This is the read-only presentation of your saved personal plan: it uses your arranged positions, colored paths, annotations, and hidden-node choices, while clicks still open ordinary lesson detail cards and drag, wheel, and pinch gestures only navigate the map.

Use the **Plan view** toggle to switch it off and inspect the untouched canonical prerequisite map. Turn it back on to return to the saved plan. Use **Plan mode** only when you want to edit that plan.

# 8. Personal Plan mode

Plan mode is the editor for the private working copy layered over the canonical mastery map. Leaving Plan mode returns to the read-only Plan view without deleting anything; the separate Plan view toggle reveals the canonical map whenever you want to compare them.

## Enter and leave Plan mode

Choose **Plan mode** in the map header. A toolbar and Plan details card appear while the map remains visible. The editable canvas extends beyond the colored field bands: those bands are reference guides, not fences, so selected nodes can be placed anywhere on the surrounding canvas. Changes autosave with your learner profile and travel in full backups and Bridge checkpoints.

## Desktop selection

- Click a node for a new selection.
- Ctrl-click adds or removes one node.
- Drag a rectangle across empty map space to select enclosed nodes.
- Hold Ctrl while box-selecting to add to the current selection.
- Drag a selected node to move the selected group.
- Shift-drag empty space to pan when selection gestures are active.

## Mobile selection

- Long-press a node to select it.
- Long-press it again to deselect it.
- Long-press empty map space to clear the selection.
- Drag selected nodes to move them.
- Drag ordinary empty space to pan.

## Hide and restore nodes

Select one or many lesson nodes and choose **Hide selected** to remove them from both the editable Plan mode and read-only Plan view. This does not delete lessons, change prerequisites, uninstall content, or affect the canonical mastery map.

Choose **Show hidden nodes** to reveal every hidden lesson as a faded, clearly labeled node. Select the ones you want back—multi-selection works normally—then choose **Unhide selected**. Choose **Hide hidden nodes** to conceal the remaining hidden nodes again.

## Custom paths

Select at least two lessons, choose **Custom path**, name the route, and choose an outline color. Selection order becomes path order. Bold connections and node outlines visualize the plan.

Custom paths do not change real prerequisites or mastery. They represent your intended study order, revision route, exam preparation, or project sequence.

## Annotations

Choose **Annotation** to create a note connected to selected lessons, or free on the map when nothing is selected. Custom paths do not own annotations; select the relevant route lessons if one comment should describe them together. Comment nodes can be dragged. Use annotations for goals, misconceptions, dates, resources, or questions. Do not store passwords or tokens in them.

## Plan details and reset

The side card lists selection, saved paths, and annotations. Delete individual plan items, hide or restore selected nodes, or reset positions on the combined map. All planning positions, free comments, paths, and hidden-node choices share the one `all-subjects` layout.

# 9. Lesson page

The lesson page is the study surface before and between assessments.

## Lesson header

The header identifies field, branch, lesson, description, preparation state, and available actions. A locked Hard-path test can still leave theory available for study.

## Theory

Theory sections explain definitions, relationships, methods, notation, and edge cases authored for the lesson. Read them actively: pause to predict the next step, restate the idea, and identify assumptions.

## Worked examples

Examples separate prompt, solution, and explanation. Try the prompt before reading the solution. Compare not only the final answer but the structure of the method.

## Applications

Applications connect the skill to real decisions, modeling, geometry, another field, or later mathematics. They explain why the lesson belongs in the map.

## Recommended preparation and unlocks

Preparation links open prerequisite lessons. Unlocks show where the current skill leads. In Open path, these remain guidance; in Hard path, they also control assessment access.

## Start mastery test

The button opens or resumes the lesson's complete assessment draft. QuickMaths uses authored scenarios rather than replacing substantial lessons with a generic five- or ten-question cap.

# 10. Mastery tests

Tests cover the authored lesson, including variations and edge cases supplied by native runtime templates or fixed validated lesson packs.

## Saved drafts

Answers and shown work autosave while you type. Returning to the same test resumes the draft. Content updates can restart affected unfinished drafts so answers never cross between different question banks.

## Question navigation

The page shows question number, total authored scenarios, prompt, response type, and any required work. Complete every visible authored scenario before submission.

## Final-answer types

Questions may accept numeric answers, tolerance ranges, multiple-choice options, sandboxed Python functions, symbolic expressions, finite sets, rational expressions with domain exclusions, equation solutions, interval sets, exact text, or theorem conclusions. Follow the notation instructions in the prompt.

Finite sets are order-independent and ignore repeated entries. You can usually write `{ -2, 5 }` or `x = -2 or x = 5`; use `{}` or `no solution` for the empty set. Interval answers accept standard interval/union notation such as `(-infinity, -1] U (3, infinity)`, and many equivalent inequality forms. Infinity endpoints must always be

open.

For a rational-expression question, the simplified formula and its excluded input values are graded together. A cancelled factor can leave a hole, so enter every original domain restriction in the separate **Excluded values** field even when that factor is no longer visible in the final formula.

## Written explanations

A separate work box can capture your reasoning even when the final answer self-grades. Explanation text is saved for Results, reflection, and optional review.

## Checked maths steps

Some advanced algebra questions require ordered steps. Put one equation, expression, or inequality per line. QuickMaths checks whether successive lines remain equivalent and whether the final line matches the answer. For inequalities, sign reversal and the full solution set matter.

This subsystem checks supported mathematical transformations. It is not a formal-proof judge.

## Rational-equation ledger

Some rational equations open a guided work area for original restrictions, algebra steps, and candidate classifications. Record values excluded by the original denominators before clearing fractions. Then enter one algebraic statement per line and classify each candidate as valid, excluded, extraneous, repeated, or non-real. QuickMaths checks the restrictions, the algebraic progression, each candidate against the original equation, and whether the valid candidates match the submitted solution set.

## Sign-chart workspace

Polynomial and rational inequalities can open a structured sign chart. Enter critical points, one test value and sign for each interval, selected intervals, endpoint decisions, and the final interval set. The page presents normal fields rather than JSON. QuickMaths reports errors by row, including a misplaced critical point, an incorrect test sign, a wrongly selected interval, or an endpoint that should be included or excluded.

## Formatted code and trace tables

Programming questions show source in a labelled code block that preserves indentation and scrolls horizontally on a narrow screen. The code displayed in a lesson prompt is inert text—it is never executed.

A structured trace question adds a scrollable table beneath the code. Each row represents a labelled execution checkpoint; enter the variable values and any output after that step. Blank output means nothing has been printed yet. QuickMaths compares the authored state table without executing the prompt code, and reports the first missing row or divergent variable/output cell before submission.

## Sandboxed Python functions

A `python_program` question provides a code editor and **Run sandboxed tests** button. Write the exact named function with the requested positional parameters and return a value instead of reading interactive input. The supported beginner subset includes assignments, conditions, loops, comprehensions, helper functions, JSON-compatible values, and an author-selected set of ordinary builtins and value methods.

The first run loads QuickMaths' self-hosted, integrity-pinned Pyodide 0.28.3 files, so it can take longer than later questions. No runtime script or package is fetched from a third-party CDN, and package installation is unavailable. Every run gets a fresh disposable background Worker. QuickMaths strictly validates the complete grading payload, rejects imports, files, network, browser/storage APIs, clocks, randomness, dynamic evaluation, private/dunder access, classes, exception handling, decorators, and unsupported syntax, and independently bounds test count, argument depth/size, source structure, execution steps, output, returned data, and wall time. Timeout or cancellation terminates the Worker; that interpreter is never reused.

Before submission, you see only tests marked as examples. After submission, additional `after_submission` outcomes can appear. Hidden tests report only status and never reveal their arguments or expected values. A current sandbox run is required before submitting; editing the code invalidates the old result. If the runtime itself cannot start, submission remains blocked and the infrastructure failure is never counted as a learner mistake. Only the visible **Run sandboxed tests** button can execute learner code; WebMCP can stage lessons but has no Python execution tool.

Python source and the bounded pass/fail summary autosave with the learner profile and can enter a full backup or complete-workspace GitHub sync. Captured stdout is transient and discarded. The subset is deliberately not full Python: file, module, exception, object-oriented, and complete application exercises remain captured or rubric-reviewed work rather than being falsely run with broad privileges. The displayed `memory_mb` authoring value is not presented as a precise browser memory quota; disposable-worker termination plus structural, data, result, and time limits are the enforceable boundary.

## Formal proof required

Proofs use ordinary text. No JSON, LaTeX, or magic keywords are required. One claim or reason per line is easiest to review.

The page shows a visible obligation checklist and accepted approaches. The short conclusion is graded separately; the proof is saved pending an obligation-by-obligation self, human, or agent review. Empty or extremely short work is blocked.

## Required long response

Rubric questions show the criteria that will be reviewed. Address each criterion with evidence, calculations, sources, or reasoning appropriate to the prompt.

## Submission boundary

Before submission, QuickMaths and its WebMCP tutor do not expose expected answers or hidden solution steps. Submit only when the final answers and required work are complete.

# 11. Results, reflection, and mastery

## Grading results

Results compare your submitted final answers with allowed grading rules. After submission, authored solutions and explanations can be shown for study. Mistake tags help identify recurring patterns.

## Work state

Each response reports whether required work was present, checked, pending review, passed, or needs revision. A correct final answer can coexist with a flawed proof or incomplete rubric response.

## Structured reflection

Record confidence, felt difficulty, hint use, guessing, desire for more practice, confusing parts, and notes. Honest reflection helps distinguish understanding from luck or fragile recall.

## Mastery update

The update combines assessment evidence, reflection, previous history, and required review state. It can move a lesson to Learning, Proven, Mastered, or later Rusty. QuickMaths does not claim mastery when a required proof or rubric review is still pending.

## Self, human, and agent review

When allowed, the Results page can save a self review. Tutor-required work needs a human tutor or connected agent.

Proof review records a status and note for every obligation: satisfied, flawed, missing, or not applicable. Rubric review records points and evidence for every criterion. The reviewer also saves concise feedback, confidence, and one next step.

## Review packet and email

Downloadable tutor summaries and review packets contain post-attempt answers, shown work, reflection, and review instructions. If the curriculum includes an educator contact, QuickMaths opens a prefilled email draft. You must attach and send the packet yourself.

## Practice again

Retaking is always available as learning practice. Native lessons can generate fresh values while covering the same authored scenarios. QuickMaths curricula do not turn practice into one-shot unsupervised exams.

# 12. Lesson Depot

Lesson Depot discovers optional fields and specialist tracks from the official catalog and independent public GitHub registries. QuickMaths merges those sources behind the same cards. Browsing, previewing, validating, and installing published packs do not require community authorization.

## Search and filters

Search matches package name, field, description, author, and tags. Filter published packages from roadmap concepts, choose field, and sort by popularity (upvotes minus downvotes), recency, or name.

## Package cards and themes

Cards use their designated field colors. They identify version, author, lesson count, tags, source, availability, and a compact provenance badge: **Official**, **Community recommended**, **New**, or **Subscribed**.

## Preview and installation safety

Published previews fetch the exact immutable package through a bounded reader, verify its registry SHA-256 hash, and run local schema, size, graph, grader, theme, and content-shape validation. If this browser cannot perform hash verification, installation stops instead of continuing unverified. You see the source and preview before confirming installation. A broken community registry is isolated and cannot prevent the official catalog or other feeds from loading.

## Community upvotes and reactions

Every federated package version is tied to one public GitHub Discussion and one exact SHA-256 digest. Lesson reactions are grouped as **Upvotes** (Heart, Rocket, Hooray, Thumbs up), **Downvotes** (Thumbs down), and **Neutral** (Eyes, Laugh, Confused). Each GitHub account contributes at most one upvote or one downvote per lesson or comment, regardless of how many emojis it adds. An account with both positive and negative reactions contributes zero to both vote totals. Neutral reactions do not cancel a vote. Popular sorting uses upvotes minus downvotes, with published packages ahead of concept previews. Valid submissions appear as **New**; a net score of three marks them **Community recommended**. Native GitHub lesson upvotes and all comment feedback do not affect lesson rankings. Downvotes do not automatically hide a package.

Comments use the same **Upvotes**, **Downvotes**, and **Neutral** reaction groups and one-vote-per-account rule as lessons. Native GitHub upvote counts are not used. Comment votes affect only that comment, not the lesson ranking. Selecting an active reaction again removes it; removing the last reaction on one side restores any remaining vote on the other side.

## Advanced lesson sources

**Settings > Manage lesson sources** shows the official catalog, automatically indexed community registries, and direct subscriptions. Paste a public GitHub registry file to subscribe immediately without waiting for global discovery. Direct feeds are marked **Subscribed**. You can remove them later or deliberately reveal contested listings. Registry subscriptions cannot read progress, profiles, backups, or storage credentials.

Validation does not prove factual correctness. Review community content for quality, licensing, suitability, and accessibility.

## Staged agent batches

An agent can stage one pack or an ordered batch for review. It cannot install. QuickMaths validates the complete ordered batch first, including dependencies on earlier packs and aggregate capacity. Settings then presents a sequential queue: you separately install or skip every package. One approval never authorizes the next.

## Upvotes and comments

Optional **Connect GitHub** authorizes the separate QuickMaths Community GitHub App for public Discussions on the QuickMaths repository. Connected humans can react to lessons or comments and post comments inside the app. These actions are public, use your GitHub identity, and are not WebMCP tools.

Community authorization is different from the fine-grained token used for private storage.

# 13. Lesson Studio for curious learners

Lesson Studio is available to everyone because explaining and authoring are powerful ways to learn. It can create new lessons or improve built-in QuickMaths lessons reversibly.

## Friendly authoring flow

Choose a field, write one or more lessons, add theory, worked examples, applications, prerequisites, and mastery questions, then validate and install. Question-mark controls provide hover, keyboard, and mobile-tap help.

## Response and review design

Studio separates final-answer grading, shown work, and review. It explains ordinary answers, finite sets, rational formulas and exclusions, interval sets, checked maths steps, rational-equation ledgers, sign charts, formatted code blocks, trace tables, sandboxed Python function tests, formal proof obligations, and rubric responses with learner previews and editable examples.

## Native improvements

Open a built-in lesson as an editable override while keeping its exact ID and field. Completed progress and map identity stay attached. Affected unfinished tests restart. Settings can restore the original.

## Human control

Validation reports issues; it does not silently install or publish. Installing requires confirmation. Lesson Depot publication opens a separate human contribution workflow.

# 14. Learning with an agent

WebMCP lets an agent inspect the same visible QuickMaths state and use registered tools only when the page is open inside the ChatGPT or Codex in-app browser. It cannot attach to an external browser tab, and it does not create a hidden parallel learner profile.

## Browser handoff and starting prompt

QuickMaths checks the WebMCP page capability rather than trusting the browser name. In an external browser with an existing workspace and no storage token, Agent Studio and tutorial chapter 6 offer **Download backup** and **Set up GitHub storage**. When private Workspace Storage is configured, **Open in ChatGPT / Codex** uses an experimental desktop deep link to open the public QuickMaths URL in the in-app browser and preload this short prompt:

```
QuickMaths is open in your in-app browser. Call get_agent_guide with section "summary" through
WebMCP, then follow the unified manifest to route me into the right learner or educator workflow.
```

The handoff URL contains no token, answers, or workspace data. Browser storage is separate, so restore the backup or enter the same fine-grained token privately in the in-app QuickMaths storage form. Credential safety, recovery rules, and detailed tutoring policy live in the manifest rather than the prompt. After the first attributed agent action, the profile remembers that an agent has participated and hides the one-time starter prompt.

On a fresh workspace, the agent first asks whether the human wants to learn or design a custom curriculum. For learning, it checks whether an existing Workspace Storage repository should be restored before asking for a new profile name. For curriculum design, it offers to help create the curriculum, guides the human to the **Educator** landing-page path, then uses `create_curriculum` only after the educator profile exists and the human approves the curriculum name and intent. After profile creation, it strongly recommends private Workspace Storage for durable cross-device recovery and agent handoff, then offers the separate optional Community connection if the human wants to upvote, react, or comment on Depot packages.

With a learner selected, the agent begins with `get_app_state`, `get_progress_summary`, and `get_learning_context` as needed. Its voice is conversational, curious, lightly quirky, and Socratic: one focused question or useful hint at a time, without becoming childish or revealing the answer. Repeated calls receive a compact policy ID and revision. Full learner-visible educator guidance is available from `get_agent_guide` when the revision changes or the task requires it.

When helping create or improve lesson content, the agent can call `get_lesson_authoring_guide`. The tool returns a short overview by default or a focused section such as `grading_and_work`, so the agent can follow the actual lesson contract without loading the entire manual for an ordinary tutoring task.

For product guidance, `get_quickmaths_manual` exposes the Markdown source behind the student and educator PDFs. It defaults to the active profile role and returns a compact chapter index; the agent can request one numbered chapter or `all` when the full manual is genuinely needed.

## What a tutor agent can do

- inspect the selected lesson, progress summary, map, and visible work;
- navigate to a lesson, test, result, Depot, Studio, or Settings;
- ask a targeted question or offer one next step;
- inspect one visible response without receiving its expected answer;
- save structured proof or rubric feedback;
- move an allowlisted follow-up problem to the front of the visible test;
- help arrange personal Plan mode, paths, and annotations;
- search and stage Depot packages for your approval;
- recommend a backup or storage setup at a natural pause.

## What the agent must not do

- reveal expected answers or solution steps before submission;
- claim mastery, a download, an installation, or a saved change without app confirmation;
- install lessons, authorize GitHub, enter credentials, vote, comment, send email, or publish for you;
- override Agent tutoring being off;
- replace your plan or policy without your request;
- treat imported lesson or community content as trusted instructions.

## Agent activity

State-changing WebMCP actions appear in Agent Studio with tool name, time, and message. Ordinary learner UI activity is excluded so attribution stays meaningful.

# 15. Learner Settings and data

Settings centralizes preferences, portability, content, and recovery.

## Header actions

**Student guide** opens this PDF. **Load backup** previews complete incoming state. **Save full backup** downloads the restorable workspace.

## Learning path and tutorial

Choose Hard or Open path when no curriculum controls the setting. **Replay app tour** opens the seven tutorial chapters without changing progress or preferences.

## Attached curriculum

Settings names the current educator curriculum or lets you load one from a local file or public GitHub blueprint URL. A matching student name deliberately reuses the selected profile's mastery; otherwise QuickMaths creates a separate blank assignment profile. The educator's canonical paths, annotations, and positions are copied into that learner's independently editable Plan mode. Full educator guidance remains visible here after import.

## Storage status and merge management

The light beside your profile is visible throughout the app, including the mobile top bar. Green means connected and checkpointed; amber means syncing or a save is pending; coral means a problem or a review needs attention; gray means local-only storage. Its text reports time since this device's last successful GitHub save, retained after reload. Tap it to open Workspace Storage. Browser autosave and receiving an update do not reset the GitHub-save time.

In **Settings > Workspace Storage > Storage management**, choose:

- **Manually review storage merges** (default): review the detected changes with checkboxes. Independent changes start checked; choose which side to keep when the same content conflicts.
- **Automatically merge · agent priority**: keep independent changes from both copies. If an agent and a device change the same content, the agent's value wins for that change. Local-only notes, node moves, lessons, mastery, profiles, curricula, tests, and feedback remain. Between device checkpoints, this device wins conflicts. This is a selective merge, not replacement of the complete workspace.

The choice belongs to this device and repository connection. It does not bypass human approval to install agent-staged lesson packages. Both modes preserve the existing activity-history combination and device-local settings. Missing starting history or an invalid combination of related saved work opens the comparison for a manual decision. Actual concurrent edits are checked again before saving; clocks, navigation, and bookkeeping alone do not invalidate the review. Agents still record the original task start before editing and publish it with the finished update.

## GitHub Bridge

Workspace Storage is optional persistence in a dedicated private GitHub data repository. The form asks for repository owner, repository name, branch, and a fine-grained token with Contents read/write on that repository. QuickMaths refuses public repositories and tokens without write access before saving the connection.

Enter the token only in the app. It is never included in backups, commits, lesson files, URLs, logs, or WebMCP results. Choose whether to keep it for this tab session or remember it in this browser.

Bridge status shows local dirty state and device label, last workspace push, the last remote writer, token storage, and remote availability. **Sync now** publishes the complete browser workspace. **Check agent updates** checks the agent checkpoint and uses your selected merge mode if work overlaps.

Workspace Storage runs across the entire app, not only while Settings is open. QuickMaths gives each browser installation a random privacy-safe device ID and a friendly label such as **Firefox on Android**; it does not read a hardware serial number. Checkpoints also identify their last writer as that device or **QuickMaths agent**.

Each agent prompt starts by noting its UTC start time locally with **begin_agent_task**. The agent pushes that original timestamp together with the finished checkpoint and its starting learner revision. If neither your device nor the GitHub learner workspace changed from that starting copy, QuickMaths applies the update automatically. If you kept working in manual mode, sync pauses and a merge window describes individual changes with checkboxes across profiles, curricula, lessons, mastery, feedback, tests, and map plans. Independent changes are checked for you. For example, keep a new GitHub note and two local node moves together. Uncheck a change to keep its starting value. For overlapping edits, choose one version or **Keep the starting value**, then **Save merged workspace**. Questions, attempts, and unfinished tests stay intact. Activity history is combined automatically. Session clocks and viewport changes alone do not cause a merge. The window works on every app page.

**Manage GitHub storage** opens the deletion manager. A profile deletion removes that profile's progress, attempts, reviews, drafts, map plan, and any educator curriculum it owns. With storage connected, QuickMaths writes the reduced learner workspace and deletes the stale agent checkpoint so it cannot remain as the current remote copy. **Clear all data** resets every local profile, curriculum, lesson pack, attempt, review, plan, Lesson Studio draft, and same-browser Agent Bridge working copy; when connected, it also deletes `learner-state.json` and `agent-state.json` from the current repository branch.

Clearing learning data preserves the configured Workspace Storage connection, remembered fine-grained token, Community authorization, and local Bridge connection. Use the separate **Disconnect** controls when you want QuickMaths to forget credentials or end a connection.

Both actions require two separate confirmations: first the full scope and backup warning, then **Are you absolutely sure?** QuickMaths has no undo. Download a JSON backup first. Deleting or replacing a current GitHub file does not erase older commits from repository history; use GitHub's repository-history controls separately if permanent historical erasure is required. If Workspace Storage is disconnected, the manager clearly limits the operation to this browser and leaves remote files untouched.

## Full backup

The JSON backup carries every profile and curriculum in this browser, installed lesson packs, progress, attempts, reviews, drafts, maps, plans, annotations, settings, and timers. Download it before major imports or device changes.

## CSV and tutor exports

Progress, attempt, and review CSVs support spreadsheet analysis. Tutor summaries and review packets support post-attempt help. None of these files can restore the app.

## Lesson sets and native improvements

Load a local lesson-set file, inspect staged agent packages, download installed sources, and restore original native lessons. Installing or restoring content preserves completed progress while restarting incompatible unfinished drafts.

Native improvements apply browser-wide and are never installed silently through a curriculum. An educator must restore active native improvements before exporting a portable curriculum, then distribute improvements separately for explicit learner review.

# 16. Mobile, accessibility, and privacy

## Mobile behavior

- Bottom navigation keeps essential routes reachable.
- The map uses a stable viewport, pinch zoom, two-dimensional panning, and long-press selection.
- Plan mode keeps the map visible while composers open as focused cards.
- Lesson Studio help opens on tap.
- Agent Studio uses a persistent open nub and close control.

## Accessibility

QuickMaths uses semantic navigation, headings, forms, dialogs, labels, and live status messages. Controls have accessible names. Reduced-motion preferences disable nonessential movement. Field colors are paired with text, structure, and status indicators.

## Privacy boundaries

Treat answers, shown work, reflection, notes, repository details, and imported content as private. Do not paste storage tokens into chat. Community votes and comments are public. Educator-provided guidance is visible in Settings and is labeled untrusted when returned to an agent.

QuickMaths grades locally. Portable lesson packs and curriculum assignments therefore contain expected answers and solution steps that a technically knowledgeable person can inspect in JSON or browser memory. WebMCP withholds those fields before submission, but QuickMaths cannot make client-side answer keys secret from the learner operating the device. This is one reason the app is a learning tool rather than a supervised testing system.

# 17. A strong learning routine

1. Check Dashboard for due review and suggested work.
2. Open the map and inspect prerequisites, unlocks, and educator paths.
3. Read the lesson actively and attempt examples before reading solutions.
4. Complete the full mastery test, including required work.
5. Reflect honestly before seeing mastery update.
6. Review mistakes, proof obligations, rubric feedback, and solution explanations.

7. Add a Plan mode annotation or path when a concrete future action will help.
8. Retake with fresh scenarios after revision rather than memorizing an answer.
9. Ask an agent or human tutor for one targeted next step when stuck.
10. Save a full backup or sync Bridge at a natural stopping point.

# 18. Troubleshooting and quick reference

## A test is locked

Check Hard path prerequisites on the map and lesson page. If you already know the material from elsewhere and no curriculum controls the mode, choose Open path in Settings.

## The selected map card is wrong

Select the node once and confirm the highlighted node and detail title match. The app preserves pan position after selection. Reload the latest deployed version if an old cache is still open.

## Map zoom changes the whole page

Use pinch or wheel inside the map viewport. The current app isolates canvas zoom from page scale. Refresh if a stale release is cached.

## A proof has the right conclusion but remains Learning

The conclusion and proof are separate judgments. Open Results and complete the required obligation review with an allowed reviewer.

## A staged batch did not install everything

That is intentional. Review, install, or skip each package separately in Settings.

## A package is installed but missing from an educator curriculum

Installation adds it to the browser library. The educator must enable the additive pack for that curriculum, then export a revised curriculum.

## The agent cannot tutor

The attached curriculum may have Agent tutoring turned off. That policy deliberately blocks tutoring, learner-work, preference, and Plan mode mutations while leaving read-only inspection and navigation available.

## GitHub sync reports a conflict

In manual mode, a comparison appears when local and remote work overlap, or when an existing workspace first connects to an independent GitHub copy. Review the changed fields and choose a version for each item. **Keep all independent changes** and **Uncheck independent changes** adjust the checklist; **Save merged workspace** applies and syncs them. **Not now** leaves sync paused. If either copy changes during review, use **Refresh comparison** before saving. If the starting copy is unavailable, every difference needs an explicit choice. GitHub history remains a recovery aid.

## WebMCP tools are unavailable

Use Agent Studio's handoff. Download a full backup or configure private Workspace Storage before leaving an existing external-browser workspace. Open QuickMaths inside the ChatGPT or Codex in-app browser, then call `get_agent_guide` with `section: "summary"`. Agent Studio names registration status and individual failures. If the in-app workspace is empty, restore the backup or enter the storage token only in the QuickMaths form, never in chat.

## Essential links

App and source: https://quickmathematics.github.io/QuickMaths/ and https://github.com/QuickMathematics/QuickMaths

Technical guides: https://quickmathematics.github.io/QuickMaths/CUSTOM_LESSON_SETS.md and https://quickmathematics.github.io/QuickMaths/bridge-guide.html